

LAIC Generalizable Coach Architecture Summary

This document summarizes the architecture discussion for building a reusable LAIC coaching platform that can support Bridge, Dance, Singing, MindAI, and future learning experiences.

The core idea is not to build one isolated coach per product. Instead, LAIC should build a reusable coaching platform where each domain plugs into a shared infrastructure through domain-specific schemas, evaluators, knowledge packages, and activity events.

1. Core Platform Idea

LAIC should be structured as a generalizable adaptive coaching platform.

The platform should support:

- Bridge AI Buddy
- MindAI Bee
- Dance Challenge
- Singing / Voice Challenge
- Future educational products

The key architecture principle is:

```
One shared platform
+ reusable common coach
+ generic adaptive coach runtime
+ domain-specific plugins
+ domain-specific knowledge packages
= scalable multi-domain coaching system
```

The system should not hardcode Bridge, Dance, or Singing into the core platform. Instead, each domain should provide its own plugin and knowledge packages while using the same common services.

2. Important Distinction: Knowledge Platform vs Domain Plugin

A major design question was whether the Domain Plugin Layer is redundant because the Knowledge Platform already has a pipeline for extracting knowledge.

The conclusion:

```
Knowledge Platform = generic factory
Domain Plugin = domain-specific instruction manual
Ingestion Intent = user's goal for the uploaded content
```

The Knowledge Platform knows **how** to process documents:

```
upload → parse → chunk → tag → extract → embed → store → review → publish
```

But it does not automatically know what “good extraction” means for Bridge, Dance, Singing, or MindAI.

The Domain Plugin tells the platform:

- What concepts exist in this domain
- What skills exist in this domain
- What event types exist during runtime
- What rule types are valid
- What validators should be used
- What evaluator should judge learner actions
- What activity state looks like
- What intervention policy should be applied

So the Knowledge Platform is domain-agnostic, while the Domain Plugin gives domain meaning.

3. Ingestion Intent: User-Guided Document Processing

Another key idea is that the platform should not blindly process uploaded documents. The frontend should ask the user:

What do you want to get from this document?

This is important because the same document can be used for different outputs.

For example, a Bridge convention document could be used to:

- Build a settings UI
- Extract bridge convention rules
- Create a configuration validator
- Create a coach knowledge package
- Generate beginner-friendly explanations
- Create a playable bot configuration

These are different goals, so the platform needs the user's intent before processing the document.

The frontend should collect an **Ingestion Intent** object.

Example:

```
type IngestionIntent = {
  intentId: string;

  domainId: string;

  documentType:
    | "lesson"
    | "rulebook"
    | "configuration_spec"
    | "assessment"
    | "activity_guide"
    | "reference"
    | "mixed";

  targetOutputs: Array<
    | "knowledge_package"
    | "experience_package"
    | "assessment_package"
    | "configuration_package"
    | "validator_package"
    | "coach_support_package"
  >;

  extractionGoals: Array<
    | "concepts"
    | "skills"
    | "rules"
    | "examples"
    | "misconceptions"
    | "hint_templates"
    | "settings"
    | "dependencies"
    | "conflicts"
    | "visibility_rules"
    | "activities"
    | "assessment_items"
    | "rubrics"
  >;

  audienceLevel: "beginner" | "intermediate" | "advanced" | "all";

  preferredUse:
    | "coach_runtime"
    | "lesson_authoring"
    | "ui_generation"
    | "engine_config"
    | "validator_generation"
    | "review_only";
```

```
humanReviewRequired: boolean;  
};
```

The final ingestion formula is:

```
Document  
+ Ingestion Intent  
+ Domain Plugin / Schema  
+ Knowledge Platform Pipeline  
= Correct Package Output
```

4. Knowledge Platform Layer

The Knowledge Platform Layer turns raw content into structured, searchable, reviewable, coach-ready packages.

It receives:

```
Raw document  
+ user ingestion intent  
+ domain schema/plugin
```

It outputs one or more packages:

- Knowledge Package
- Configuration Package
- Validator Package
- Experience Package
- Assessment Package
- Coach Support Package

Knowledge Platform Components

Knowledge Platform Layer

- Document Upload Service
 - └ receives PDF / DOCX / Markdown / text / transcript
- Ingestion Intent Builder
 - └ asks what the user wants from the document
- Domain Schema Loader
 - └ loads selected domain schema/plugin
- Universal Document Parser
 - └ extracts text, headings, tables, lists, JSON/code blocks, source references
- Universal Chunker
 - └ chunks using document structure + schema + intent
- Metadata Tagger
 - └ tags chunks with domainId, conceptIds, skillIds, eventTypes, activityTypes, rules
- Concept / Skill / Rule Extractor
 - └ extracts requested objects based on ingestion intent
- Package-Type Builder
 - └ creates correct package drafts
- Embedding Generator
 - └ embeds retrievable chunks
- Vector Store
 - └ stores searchable chunks
- Structured Knowledge Store
 - └ stores exact concepts, skills, rules, settings, dependencies, conflicts, examples
- Human Review Interface
 - └ lets humans approve, edit, reject, and publish extracted knowledge
- Knowledge Package Manifest Builder
 - └ publishes final package for runtime use

Why this layer matters

The coach should not directly read raw PDFs or long documents during runtime. Instead, the documents should first be converted into:

1. Structured knowledge
exact rules, settings, dependencies, conflicts, concepts, skills
2. Searchable knowledge
chunks, examples, explanations, source passages
3. Package manifests
versioned links to source documents, vector indexes, rules, evaluators, and retrieval

Structured knowledge tells the system what is true. Searchable knowledge helps the LLM explain why it matters.

5. Where the LLM Comes In

The LLM is important, but it should not be the whole system.

Use deterministic code for:

- File upload
- Document parsing when structure is clear
- Schema loading
- Validation
- Storage
- Package manifest creation
- Exact rule enforcement
- Dependency checking
- Conflict checking
- Legal action checking

Use the LLM for:

- Intent suggestion
- Semantic chunking when the document is messy
- Metadata tagging

- Concept / skill / rule extraction
- Beginner-friendly explanations
- Hint generation
- Postmortem summaries
- Runtime coaching responses

Pipeline with LLM placement:

Document Upload Service

LLM: No

Ingestion Intent Builder

LLM: Optional, suggests document purpose

Domain Schema Loader

LLM: No

Universal Document Parser

LLM: Usually no

Universal Chunker

LLM: Sometimes, for semantic re-chunking

Metadata Tagger

LLM: Yes

Concept / Skill / Rule Extractor

LLM: Yes

Package-Type Builder

LLM: Optional

Human Review Interface

LLM: Optional assistant

Embedding Generator

LLM: Embedding model, not chat LLM

Vector Store

LLM: No

Structured Knowledge Store

LLM: No

Knowledge Package Manifest Builder

LLM: Usually no

Adaptive Coach Runtime

LLM: Yes, for final coaching response

The safest rule:

LLM extracts and explains.
Validators enforce.
Humans approve.
Structured stores preserve.
Adaptive coach responds.

6. Domain Plugin / Schema Layer

The Domain Plugin Layer makes the platform generalizable.

It is separate from the Knowledge Platform because the Knowledge Platform knows the generic process, but the plugin knows the domain-specific meaning.

Domain Plugin Components

```
Domain Plugin / Schema Layer
|
├── Bridge Gameplay Plugin
|   ├── bridge_gameplay.schema.json
|   ├── bridge_gameplay.events.ts
|   ├── bridge_gameplay.activity_state.ts
|   ├── bridge_gameplay.validators.ts
|   ├── bridge_gameplay.evaluator.ts
|   └── bridge_gameplay.intervention_policy.json
|
├── Bridge Config Plugin
|   ├── bridge_config.schema.json
|   ├── bridge_config.events.ts
|   ├── bridge_config.activity_state.ts
|   ├── bridge_config.validators.ts
|   ├── bridge_config.evaluator.ts
|   └── bridge_config.intervention_policy.json
|
├── Dance Plugin
├── Singing Plugin
├── MindAI Plugin
└── Future Domain Plugins
```

The plugin provides:

1. Domain vocabulary
2. Skill taxonomy

3. Event schema
4. Activity state schema
5. Rule types
6. Validators
7. Evaluator configuration
8. Intervention policy

Example: Bridge Gameplay Plugin

```
{
  "domainId": "bridge_gameplay",
  "conceptCategories": [
    "bidding",
    "hand_evaluation",
    "declarer_play",
    "defense",
    "counting",
    "planning",
    "conventions",
    "communication"
  ],
  "eventTypes": [
    "deal_started",
    "bid_made",
    "card_played",
    "trick_completed",
    "hint_requested",
    "hand_completed"
  ],
  "ruleTypes": [
    "bidding_rule",
    "convention_rule",
    "play_principle",
    "defense_principle"
  ],
  "evaluator": {
    "type": "engine",
    "name": "BEN"
  }
}
```

Example: Bridge Config Plugin

```

{
  "domainId": "bridge_config",
  "conceptCategories": [
    "settings",
    "control_types",
    "modules",
    "presets",
    "value_profiles",
    "visibility_profiles",
    "dependencies",
    "conflicts",
    "mutual_exclusivity",
    "profile_resolution"
  ],
  "eventTypes": [
    "preset_selected",
    "setting_changed",
    "setting_search",
    "dependency_evaluated",
    "conflict_detected",
    "visibility_profile_applied",
    "profile_saved"
  ],
  "ruleTypes": [
    "control_type_rule",
    "dependency_rule",
    "mutual_exclusivity_rule",
    "conflict_warning_rule",
    "visibility_rule",
    "resolution_rule"
  ],
  "evaluator": {
    "type": "rules",
    "name": "bridge_config_validator"
  }
}

```

7. Common Platform Layer

The Common Platform Layer contains shared infrastructure used by all products.

It should not contain Bridge-specific, Dance-specific, or Singing-specific rules.

```
Common Platform Layer
|
├─ User Profile Service
├─ Learner Model Service
├─ Session Engine
├─ Progress Tracking
├─ Achievement / Badge Service
├─ Recommendation Engine
├─ Reflection Engine
├─ Planner Engine
├─ Dashboard Framework
└─ Analytics Service
```

This layer answers:

- Who is the learner?
- What have they completed?
- What are their strengths and weaknesses?
- What session are they in?
- What should they do next?

8. Common Coach Layer

The Common Coach is the long-term cross-domain learning coach.

It is not the same as the real-time adaptive coach.

The Common Coach creates a universal learner context package that any adaptive coach can use.

```
Common Coach Layer
|
├─ Learner Summary Generator
├─ Learning Goal Resolver
├─ Weak Skill Detector
├─ Feedback Style Resolver
├─ Practice Plan Generator
├─ Reflection Prompt Generator
├─ Recommendation Context Builder
└─ Common Coach Package Builder
```

Example output:

```

type CommonCoachPackage = {
  learner: {
    learnerId: string;
    skillLevel: "beginner" | "intermediate" | "advanced";
    preferences: {
      feedbackStyle: "gentle" | "direct" | "socratic" | "minimal";
      explanationDepth: "short" | "medium" | "deep";
      interruptionTolerance: "low" | "medium" | "high";
    };
  };

  learningState: {
    currentProductId: string;
    currentDomainId: string;
    currentExperienceId: string;
    currentActivityId: string;
    currentLearningGoal: string;
    masteredSkills: string[];
    weakSkills: string[];
    recentMistakes: string[];
    recentFeedbackSummary: string;
  };

  coachingPolicy: {
    maxHintLevel: 1 | 2 | 3 | 4;
    allowDirectAnswer: boolean;
    allowRealTimeInterruption: boolean;
    saveForPostmortemWhenPossible: boolean;
  };
};

```

For Bridge, the Common Coach may say:

```

Learner is beginner.
Current goal is opening bids.
Weak skill is opening bid selection.
Use short hints.
Do not reveal direct answers immediately.

```

For Dance, the same package shape may say:

Learner is beginner.
Current goal is staying on beat.
Weak skill is timing.
Use short corrections.
Interrupt only after repeated error.

9. Adaptive Coach Runtime Layer

The Adaptive Coach Runtime is the real-time coaching engine.

It is generic. It becomes Bridge, Dance, Singing, or MindAI by receiving different domain plugins, packages, activity states, evaluators, and event types.

```
Adaptive Coach Runtime Layer
|
├─ Generic AdaptiveCoach Object
├─ Event Listener
├─ Context Package Builder
├─ Knowledge Retriever
├─ Domain Evaluator Adapter
├─ Diagnosis Engine
├─ Intervention Policy Engine
├─ LLM Response Generator
└─ Session Update Layer
```

Runtime formula:

```
CommonCoachPackage
+ KnowledgePackage
+ LiveActivityEvent
+ ActivityState
+ DomainEvaluatorResult
+ LLM
= AdaptiveCoachResponse
```

Generic runtime input:

```

type AdaptiveCoachRuntimeInput<TActivityState, TAction> = {
  common: CommonCoachPackage;
  knowledgePackageId: string;

  event: {
    eventId: string;
    eventType: string;
    timestamp: string;
    actorId: string;
    action: TAction;
  };

  activityState: TActivityState;
};

```

Generic runtime output:

```

type AdaptiveCoachResponse = {
  type:
    | "silent"
    | "nudge"
    | "hint"
    | "explanation"
    | "warning"
    | "postmortem_note"
    | "profile_summary";

  level?: 1 | 2 | 3 | 4;
  message?: string;

  metadata?: {
    relatedConceptIds?: string[];
    relatedSkillIds?: string[];
    sourceChunkIds?: string[];
    savedForPostmortem?: boolean;
  };
};

```

Bridge gameplay example:


```
bid_made event
↓
retrieve opening bid knowledge
↓
BEN evaluates action
↓
diagnose learner mistake
↓
LLM gives hint
```

Bridge configuration example:

```
setting_changed event
↓
retrieve conflict/dependency rules
↓
config validator evaluates profile
↓
diagnose conflict
↓
LLM explains warning
```

10. Domain Evaluator Layer

The Domain Evaluator judges whether an action is valid, good, weak, problematic, or incorrect.

The LLM should not be the sole evaluator.

```
Domain Evaluator Layer
|
├── Bridge Gameplay Evaluator
│   └── BEN / Bridge engine
|
├── Bridge Config Evaluator
│   └── Configuration validator
|
├── Dance Evaluator
│   └── Timing / movement / video analyzer
|
├── Singing Evaluator
│   └── Pitch / rhythm / audio analyzer
|
├── MindAI Evaluator
│   └── Quiz / scenario grader
|
└── Generic Rule Evaluator
    └── Deterministic rule checks
```

The evaluator gives the adaptive coach judgment. The LLM gives the learner an explanation.

11. LLM / AI Orchestration Layer

This layer manages model calls, prompts, tool use, RAG context, and response formatting.

```
LLM / AI Orchestration Layer
|
├── Prompt Template Manager
├── RAG Context Builder
├── Tool Calling Controller
├── Response Generator
├── Safety / Grounding Guardrails
├── Output Formatter
├── Citation / Source Attacher
└── Model Router
```

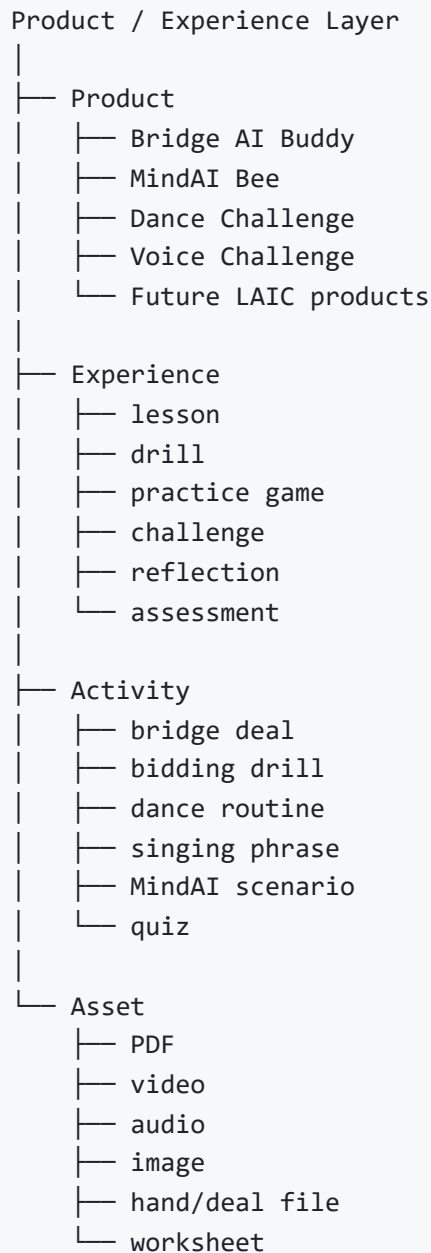
At runtime, the LLM receives:

```
learner context
+ retrieved knowledge
+ domain evaluator result
+ current event
+ intervention decision
+ output policy
```

Then it produces learner-facing coaching language.

12. Product / Experience Layer

This layer defines what the learner is doing.



This layer answers:

- What product is the learner in?
 - What experience is active?
 - What activity is active?
 - What skill is this activity supposed to teach?
 - What assets does the activity use?
-

13. Data / Memory Layer

This layer stores long-term and short-term state.

```
Data / Memory Layer
|
├─ User Store
├─ Learner Profile Store
├─ Learner Skill State Store
├─ Session Store
├─ Event Log Store
├─ Coach Interaction Store
├─ Knowledge Package Store
├─ Vector Store
├─ Structured Knowledge Store
├─ Domain Schema Store
├─ Experience Store
├─ Activity Store
├─ Review Store
└─ Analytics Store
```

Important tables:

```
users
learner_profiles
learner_skill_states
sessions
session_events
coach_interactions
knowledge_packages
knowledge_chunks
concepts
skills
rules
experiences
activities
domain_schemas
review_items
```

This layer remembers:

- What the learner did
- What feedback they received
- What skills improved

- What knowledge package was used
 - Which source supported the answer
 - Which rules were approved
-

14. Admin / Review Layer

This layer allows humans to review and control what the platform publishes.

```
Admin / Review Layer
|
├─ Domain Schema Editor
├─ Ingestion Intent Review
├─ Chunk Review
├─ Metadata Review
├─ Extracted Rule Review
├─ Concept / Skill Review
├─ Package Publishing Workflow
├─ Version Control
├─ Rollback Tools
└─ Quality Dashboard
```

This is important because extracted knowledge can affect real coaching behavior. For serious educational use, the system should not automatically publish all AI-extracted rules without review.

15. UI / Delivery Layer

This is where the learner sees and interacts with the coach.

UI / Delivery Layer

- |
- | — Bridge Table UI
 - | — coach bubble
 - | — coach sidebar
 - | — hint button
 - | — postmortem screen
 - | — coach on/off toggle
- |
- | — Bridge Settings UI
 - | — setting explanations
 - | — conflict warnings
 - | — disabled control explanations
 - | — profile summary
- |
- | — Dance Practice UI
 - | — timing feedback
 - | — routine progress
 - | — correction prompts
- |
- | — Singing Practice UI
 - | — pitch feedback
 - | — rhythm feedback
 - | — phrase-level coaching
- |
- | — MindAI Lesson UI
 - | — tutor chat
 - | — scenario feedback
 - | — quiz explanation
 - | — reflection prompt

16. Example End-to-End Pipeline: Bridge Config Document

Suppose the user uploads:

Bridge Convention Configuration – UI Specification v3

The frontend asks:

What do you want from this document?

The user selects:

Domain: Bridge Configuration

Document type: Configuration Specification

Target outputs:

- Configuration Package
- Validator Package
- Coach Support Package

Extract:

- settings
- modules
- control types
- presets
- dependency rules
- conflict warnings
- visibility rules
- profile resolution logic

Preferred use:

- settings UI
- adaptive configuration coach

The pipeline:

Document Upload Service
↓
Ingestion Intent Builder
↓
Domain Schema Loader loads bridge_config plugin
↓
Universal Document Parser extracts headings, tables, JSON examples, rule lists
↓
Universal Chunker chunks by module, setting table, dependency section, conflict section,
↓
Metadata Tagger labels chunks with setting keys, event types, rule types, package types
↓
Concept / Skill / Rule Extractor extracts settings, dependencies, conflicts, visibility r
↓
Package-Type Builder creates configuration, validator, and coach-support package drafts
↓
Human Review Interface approves extracted rules
↓
Embedding Generator embeds approved chunks
↓
Vector Store stores searchable chunks
↓
Structured Knowledge Store stores exact settings/rules/profiles
↓
Knowledge Package Manifest Builder publishes bridge_config_v3
↓
Bridge settings UI sends real-time setting_changed event
↓
Adaptive Coach retrieves relevant rules/chunks
↓
Config Validator checks conflict/dependency
↓
LLM explains result to learner
↓
Coach UI displays feedback

Example runtime:

User enables Texas transfers while Jacoby transfers are off.

The config validator detects a non-blocking conflict.

The LLM explains:

Texas transfers usually work together with Jacoby transfers. You can keep this setting, b

17. Final Main Architecture

This is the final high-level architecture for the LAIC generalizable coach system.

LAIC Generalizable Coach Architecture

— 1. Product / Experience Layer

- Products: Bridge AI Buddy, MindAI Bee, Dance, Singing, future apps
- Experiences: lessons, drills, games, challenges, reflections, assessments
- Activities: bridge deals, bidding drills, dance routines, singing phrases, scenarios
- Assets: PDFs, videos, audio, images, deal files, worksheets

— 2. Common Platform Layer

- User Profile Service
- Learner Model Service
- Session Engine
- Progress Tracking
- Achievement / Badge Service
- Recommendation Engine
- Reflection Engine
- Planner Engine
- Dashboard Framework
- Analytics Service

— 3. Common Coach Layer

- Learner Summary Generator
- Learning Goal Resolver
- Weak Skill Detector
- Feedback Style Resolver
- Practice Plan Generator
- Reflection Prompt Generator
- Recommendation Context Builder
- Common Coach Package Builder

— 4. Knowledge Platform Layer

- Document Upload Service
- Ingestion Intent Builder
- Domain Schema Loader
- Universal Document Parser
- Universal Chunker
- Metadata Tagger
- Concept / Skill / Rule Extractor
- Package-Type Builder
- Embedding Generator
- Vector Store
- Structured Knowledge Store
- Human Review Interface
- Knowledge Package Manifest Builder

— 5. Domain Plugin / Schema Layer

- Bridge Gameplay Plugin
- Bridge Config Plugin
- Dance Plugin

- | — Singing Plugin
- | — MindAI Plugin
- | — Future Domain Plugins
- |
- | — 6. Adaptive Coach Runtime Layer
 - | — Generic AdaptiveCoach Object
 - | — Event Listener
 - | — Context Package Builder
 - | — Knowledge Retriever
 - | — Domain Evaluator Adapter
 - | — Diagnosis Engine
 - | — Intervention Policy Engine
 - | — LLM Response Generator
 - | — Session Update Layer
- |
- | — 7. Domain Evaluator Layer
 - | — Bridge Gameplay Evaluator: BEN / Bridge engine
 - | — Bridge Config Evaluator: configuration validator
 - | — Dance Evaluator: timing / movement analyzer
 - | — Singing Evaluator: pitch / rhythm analyzer
 - | — MindAI Evaluator: quiz / scenario grader
 - | — Generic Rule Evaluator
- |
- | — 8. LLM / AI Orchestration Layer
 - | — Prompt Template Manager
 - | — RAG Context Builder
 - | — Tool Calling Controller
 - | — Response Generator
 - | — Safety / Grounding Guardrails
 - | — Output Formatter
 - | — Citation / Source Attacher
 - | — Model Router
- |
- | — 9. Data / Memory Layer
 - | — User Store
 - | — Learner Profile Store
 - | — Learner Skill State Store
 - | — Session Store
 - | — Event Log Store
 - | — Coach Interaction Store
 - | — Knowledge Package Store
 - | — Vector Store
 - | — Structured Knowledge Store
 - | — Domain Schema Store
 - | — Experience Store
 - | — Activity Store
 - | — Review Store
 - | — Analytics Store

- └─ 10. Admin / Review Layer
 - └─ Domain Schema Editor
 - └─ Ingestion Intent Review
 - └─ Chunk Review
 - └─ Metadata Review
 - └─ Extracted Rule Review
 - └─ Concept / Skill Review
 - └─ Package Publishing Workflow
 - └─ Version Control
 - └─ Rollback Tools
 - └─ Quality Dashboard
- └─ 11. UI / Delivery Layer
 - └─ Bridge Table UI
 - └─ Bridge Settings UI
 - └─ Dance Practice UI
 - └─ Singing Practice UI
 - └─ MindAI Lesson UI
 - └─ Future Product UIs

18. Final Mental Model

Common Coach = knows the learner.
Knowledge Platform = prepares the material.
Domain Plugin = defines the domain meaning.
Domain Evaluator = judges the learner action.
Adaptive Coach = decides what coaching should happen.
LLM = explains it well.
UI = delivers it to the learner.
Data Layer = remembers everything.
Admin Layer = keeps quality under human control.

The most important overall formula:

Schema + Content + Ingestion Intent + Learner Model + Real-Time Events + Evaluator
= Generalizable Adaptive Coaching Platform